application. Users should be able to authenticate with their username and password through a login form. When we configure Spring Security to enable such a feature, the framework adds a filter that intercepts any incoming HTTP request. If the user is already authenticated, it sends the request through to be processed by a given web handler, such as a `@RestController` class. If the user is not authenticated, it forwards the user to a login page and prompts for their username and password.

NOTE In imperative Spring applications, a filter is implemented as a Servlet `Filter` class. In reactive applications, the `WebFilter` class is used.

Most Spring Security features, when enabled, are handled via filters. The framework establishes a chain of filters that are executed according to a well-defined and sensible order. For instance, the filter that handles authentication runs before the one that checks for authorization because we can't verify a user's authority before knowing who it is.

Let's start with a basic example to better understand how Spring Security works. We want to add authentication to the Polar Bookshop system. Since Edge Service is the entry point, it makes sense to handle cross-cutting concerns like security there. Users should be able to authenticate with a username and password via a login form.

First, add a new dependency on Spring Security in the build.gradle file for the Edge Service project (edge-service). Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 11.1 Adding dependency for Spring Security in Edge Service

```
dependencies {
  ...
  implementation 'org.springframework.boot:spring-boot-starter-security'
}
```